# Lab 2 bonus

3240104505 查沣翊 (Grapesea)

## 1 Bonus: 在RISC-V上进行优化

嗯，这部分我其实没怎么动手，一方面是我累坏了，另一方面是RISCV环境比较诡异. 以下基本由Deepseek-V4-Flash生成过程:

### 1.1 环境配置

在 RISC-V 集群节点上跑 `1.sh`，日志里出现两类错误:

```
c++: error: '-march=sapphirerapids': ISA string must begin with rv32 or rv64
...
1.sh: 15: ./build/lab2: Exec format error
```

`Exec format error` 对应退出码 126，是 shell 尝试执行**错误架构的可执行文件**时才返回的错误，不是我的代码运行崩溃. 顺着日志追了一下，是这条失败链:

1. `build/CMakeCache.txt` 是在 x86_64 机器上生成的，cache 里缓存了 `CMAKE_SYSTEM_PROCESSOR=x86_64` 和已经通过的 `COMPILER_SUPPORTS_SPR`;
2. RISC-V 节点上 cmake 复用了这份 stale cache: `CMakeLists.txt` 的 `-march=sapphirerapids` 分支被触发，RISC-V 的 g++ 直接拒绝该参数，`student` 目标编译失败;
3. 脚本不管编译失败继续执行 `./build/lab2`，此时运行的是 build 目录里残留的 x86-64 ELF 二进制，RISC-V 内核加载不了，于是 `Exec format error`，退出码 126.

所以根因是跨机器/跨架构复用了 build 目录，和算法本身一点关系都没有. 修复也很简单，`../1.sh` 在 cmake 配置前先清空再生成:

```
cmake -E remove_directory build
cmake -B build
```

第二个坑是代码两头都编译不过: `student/moe_opt.cpp` 里 342 处 `_mm512*`、112 处 `_tile_*`（AMX）、`SYS_arch_prctl`/`XFEATURE_XTILEDATA` 权限申请，任何一处都过不了 RISC-V 的 g++; 而文件顶部又无条件 `#include <riscv_vector.h>`，x86 的 g++ 也编译不过——当时文件处于"两头都编译不了"的半移植状态. 我的做法是用一个宏把整段 x86 实现包起来，**原实现一行不动**:

```
#if (defined(__x86_64__) || defined(__amd64__)) && !defined(MOE_FORCE_PORTABLE)
    // ... 约 2040 行的 AVX-512/AMX 实现
#else
    // RISC-V 可移植后端（标量 + RVV/IME）
#endif
```

同时加了一个编译期开关 `MOE_FORCE_PORTABLE`: 在任何机器上定义它就会强制走非 x86 分支，这样我可以在 x86 主机上直接验证 RISC-V 后端的正确性（四个测试 rel RMSE = 0，与参考实现**逐位一致**），不用交叉编译也能先把逻辑错误排除掉. x86 专属头文件（`immintrin.h`、`linux/futex.h` 等）也全部移进 `#if` 分支内，RISC-V 的 g++ 从此不需要看见任何 x86 的东西.

第三个坑是 walltime: 框架 `main.cpp` 里参考实现的循环默认跑 1000 次迭代，Test 4 的 baseline 在 x86 上就要约 230 s，RISC-V 节点只会更慢，仅 baseline 就远超作业申请的 walltime. 好在 `main.cpp` 支持第 6 个参数指定迭代次数，`1.sh` 给四个测试显式传了较小的迭代（200/200/100/10），正确性检查照常执行，整个脚本 13 分钟以内能跑完.

纯标量移植版（`riscv_forward_scalar`：和参考实现完全一致的公式 + 按 token 的 OpenMP 并行）在 RISC-V 节点上的结果见 `results/20260803_085912.log`：S1 4.06x、S2 4.84x、S3 4.20x、S4 2.55x. 多 token 场景勉强能用，单 token 场景和 x86 上 OpenMP 阶段的表现一模一样——只有并行没有向量化，天花板就在那.

## 1.2 热点分析

先按计算量把四个场景分个类（D=输入维，H=FFN 维，E=专家数，K=top-k）：

```
router:   E·D 次 fp32 MAC + E 次 expf (sigmoid)
专家:     (K+1) 个专家 × 3DH 次 int8 MAC (gate + up + down)，中间含 H 次 expf 和两次量化
```

| 测试 | 形状 | router MAC | 专家 MAC | 主导瓶颈 |
|---|---|---|---|---|
| 1 | N=1, D=256, H=128, E=16, K=4 | 4K | 0.49M | 延迟（单 token，逐专家小 GEMM） |
| 2 | N=1, D=1024, H=512, E=16, K=4 | 16K | 7.9M | 延迟（GEMM 为主） |
| 3 | N=128, D=256, H=128, E=16, K=4 | 0.52M | 63M | 计算（可并行） |
| 4 | N=1024, D=512, H=128, E=512, K=2 | 268M | 604M | 内存带宽 + 计算 |

结论和主线任务几乎一样，甚至更极端：

1. **专家 GEMM（int8×int8→int32）是绝对热点**，Test 3/4 里占 90% 以上计算量. 标量基线每个 MAC 一条指令；向量化之后每指令 16 个 MAC（VLEN=256 的 16-bit 数据通路全宽利用），这是最大的杠杆；
2. **Test 4 是内存带宽敏感**：baseline 每个 token 都把全部专家权重重读一遍（约 604 MB/轮），打包 + 按专家分桶后权重按 token 组复用，DRAM 流量降约 8 倍到 ~75 MB/轮；
3. **router 的 expf 是隐藏热点**：Test 4 每轮 1024×512 = 52 万次标量 expf，向量 sigmoid（16 lane 多项式）把它变成几万条向量指令；
4. **S1/S2 是延迟敏感**：单 token、专家又小，GEMM 大不起来，要做的是提高单条指令的 MAC 数，同时别引入线程/上下文切换开销.

等 IME 内核写完，我又用 gprof 单独采了 S2 一轮热点（`-pg` 构建，只跑 S2 路径 1000 轮；`init_tokens`/`preprocess` 在计时区外不算）：

| 热点 | flat profile 占比 | 含义 |
|---|---|---|
| `ime_vmadot_tile` | 63.25% | 单 token Gate/Up/Down 的 IME K 循环 |
| `ime_gemm_packed_single` 调用子树 | 约 16.4% | output tile 遍历和 IME leaf 调用 |
| `rvv_expert_ffn` self | 1.53% | SwiGLU、量化和最终写回 |
| `rvv_prepare_block` | 0.22% | Router、Top-K 和输入量化 |

这个结果很关键：Router 在 S2 里只剩 0.22%，说明继续抠 sigmoid 近似或优化 Top-K 最多影响零点几个百分点，正确方向应当是单 token 投影和 `vmadot` leaf 本身. 后面做的软件流水正是从这里来的.

## 1.3 RVV 向量化优化（VLEN 无关的设计思路）

RVV 和 AVX-512 的第一个区别是设计哲学. AVX-512 把宽度焊死在 ISA 里（SSE 128 → AVX 256 → AVX-512 512，每代换一套 intrinsic）；RVV 把"这次算多宽"从指令里剥离出来，由 `vsetvli` 在运行时决定，代码只写 SEW（元素位宽）和 LMUL（寄存器组倍数），同一份代码可以适配不同硬件实现的 VLEN：

| 特性 | x86 Sapphire Rapids | RISC-V Muse Pi Pro (K1) |
|---|---|---|
| 向量宽度 | AVX-512 固定 512-bit；AMX 64B tile | VLEN=256，由硬件决定 |
| 编程模型 | 定长寄存器、固定 tile 指令（`_tile_dpbssd`） | SEW/LMUL 运行时参数，VLEN 无关代码 |
| int8→int32 GEMM | AMX `_tile_dpbssd` 一条指令完成 16×64 MAC | 没有矩阵单元；`vwmacc.vx` 是**加宽**乘加：int8 输入只能累加到 int16，必须先 `vsext` 扩到 int16 才能累加到 int32 |
| 乘减语义 | `_mm512_fnmadd` = a − b·c | `vfnmacc` 对整个和取负（−(vd+rs1·vs2)）；"减法"要用 `vfnmsac` |
| 饱和窄化 | `_mm512_cvtsepi32_epi8` 一步完成 | `vnclip` 一次只砍半位宽（i32→i16→i8 需两步） |
| 数据布局 | AMX 16×64B 行主序 tile | 无固定 tile；为让 16 个输出行连续可向量加载，需在 preprocess 里按 16 行块转置 |

基于这些差异，我的设计是 **16 条 lane 的固定工作宽度**：VLEN=256 时 16 个 int16 恰好占满一个向量寄存器，16 个 int32 累加器占 LMUL=2. int8 权重在 `preprocess` 里按 16 输出行一块转置打包成 `[fb][k][16]`（`pack_gu_tiles`，位于`../student/moe_opt.cpp`）：

```
// dst[(b,k)][lane] = w[b*16+lane][k]: 同一 k 的 16 个输出行连续
for (int b = 0; b < fb; ++b)
    for (int k = 0; k < cols; ++k)
        for (int l = 0; l < RVV_FB; ++l)
            dst[(size_t)(b * cols + k) * RVV_FB + l] =
                gate[(size_t)(b * RVV_FB + l) * cols + k];
```

这样一次 `vle8` 就是一条能直接喂给 `vwmacc` 的向量. GEMM 微内核（`gemm_tile_s8_live`，位于`../student/moe_opt.cpp`）长这样：

```
for (int k = 0; k < k_dim; ++k) {
    vint8mf2_t w8 = __riscv_vle8_v_i8mf2(w + (size_t)k * RVV_FB, vl);
    vint16m1_t w16 = __riscv_vsext_vf2_i16m1(w8, vl);  // 符号扩展一次
    a0 = __riscv_vwmacc_vx_i32m2(a0, xq[rows[0]*k_dim + k], w16, vl);
    a1 = __riscv_vwmacc_vx_i32m2(a1, xq[rows[1]*k_dim + k], w16, vl);
    // ... 8 个 int32 累加器
}
```

几个关键点：

- `vwmacc.vx` 把 int16 权重向量与标量 xq 相乘并累加到 int32，**一条指令 16 个 MAC**，而标量基线是一个 MAC 一条指令；
- 权重向量只加载/扩展一次，被 **8 个 token 累加器共享**（8 个 `vint32m2_t` 占 16 个向量寄存器，加上权重和临时变量，32 个寄存器内不溢出）——每条权重字节被 8 个 token 复用，这是 Test 4 带宽降 8 倍的直接原因；
- GCC 不允许 "RVV 类型数组"，8 个累加器用命名变量 + 指针数组绕过去；token 数不足 8 时按 `LIVE` 模板分派（`if constexpr`，1..8），多余的累加器不生成指令.

踩坑记录：

1. `vfnmacc` **语义与 x86 不同**：RVV 的 `vfnmacc = -(vd + rs1·vs2)`，不是 "a − b·c". 参数归约 `x - fx·ln2` 必须用 `vfnmsac`（= vd − rs1·vs2）；
2. **多项式是 Horner 形** `y = y·x + c`（`vfmacc_vv(c, y, x)`），不是 `y + c·x`；写反会导致 exp(0)=1.5 这类错误；
3. **silu(v) = v·σ(v)**，第一次实现漏乘了 vg，隐藏层数值整体错掉；
4. **gate 双重计数**：down 里已经乘了 gate，combine 里又乘一次（输出恰为参考的 gate 倍），删除 combine 的乘加、改为直接加；

5. **打包布局与 GEMM 步长不一致**：gate/up 交错 32 字节存 → GEMM 按 16 字节读，结果错乱，改为 gate/up 各占一半连续区.

另外坦白一下：我并没有把代码写成真正 "VLEN 无关"——16 lane 是写死的，但 `preprocess` 里在运行时检查 `__riscv_vsetvlmax_e8m1() >= 32`（即 VLEN≥256）且 d_model/d_ff 是 16 的倍数，不满足就整体回退标量后端. RVV 把"硬件实现多宽"的自由留给厂商，我用运行时检查兜底保证自己的假设成立，这也算一种妥协的移植策略.

## 1.4 IME 矩阵扩展（4×4×8 的 vmadot）

IME（进迭时空的矩阵扩展）是 RISC-V 生态里厂商扩展的典型样本：在标准 RVV 之外用自定义编码加矩阵指令. 它的 `vmadot` 固定计算一个 $A[4][8] \times B[4][8]^T \rightarrow C[4][4]$ 的 signed int8 tile（4 个 token × 4 个输出通道 × 8 个归约元素），和 AMX 的 16×64 大 tile 形成鲜明对比. 好处是它直接消费普通的向量寄存器（v0/v1/v2...），复用 RVV 的加载指令，不需要 AMX 那套 tile load/store 和 `arch_prctl` 权限申请；代价是工具链不认识——binutils 早于厂商助记符，只能手写编码：

```
.word 0xe2103e2b    # vmadot v28, v0, v1
```

权重在 `preprocess` 里打包成 `[output_block][k_block][4][8]`，一次 `vle8` 32 字节正好一个 K 块. 多 token 路径用双输出版本：一次扫两个 output block，v26/v28 两个累加器并行，把 S3 常见的 16-token batch 从四次相距很远的权重遍历变成紧挨着的四次复用.

单 token（S1/S2）则是另一回事：`vmadot` 的 M 方向固定 4 行，单 token 只有 1 行有效，矩阵利用率只有 25%. 我先验证了"IME 浪费 3 个 M 行，RVV 单 token 内核可能更快"这个猜想，把 Gate/Up 和 Down 分别换成 RVV 做 A/B（300 轮、同一作业内三次取中位）：

| Gate/Up | Down | 三次优化时间 | 中位数 |
|---|---|---|---|
| IME | IME | 0.493 / 0.471 / 0.471 s | 0.471 s |
| RVV | IME | 0.511 / 0.523 / 0.531 s | 0.523 s |
| IME | RVV | 0.531 / 0.503 / 0.500 s | 0.503 s |
| RVV | RVV | 0.537 / 0.520 / 0.519 s | 0.520 s |

结果很有意思：虽然 IME 在 M 方向只有 25% 结果有效，但一条 `vmadot` 仍然比 RVV 的 `vle8 + vsext + vwmacc` 指令流更有优势——RVV 每 16 个 MAC 要 3 条指令，IME 是 load + 一条矩阵指令. Gate/Up 改 RVV 回退约 11%，Down 改 RVV 回退约 7%，因此最终保留全 IME.

针对 S2 我还在这个 leaf 上做了最后一点优化——**软件流水**. 原来的每个 K 块严格执行 `vle8 A -> vle8 B -> vmadot -> 指针递增`，`vmadot` 紧跟在两次 load 后面，会直接暴露向量 load-to-use 延迟. 改成两组输入寄存器交替：当前 `vmadot` 消费 v0/v1 的时候，v2/v3 已经在下一次 load 里准备好了，让 load 延迟和矩阵计算重叠：

```
load block 0 -> v0/v1
load block 1 -> v2/v3
vmadot v28, v0, v1
load block 2 -> v0/v1
vmadot v28, v2, v3
...
```

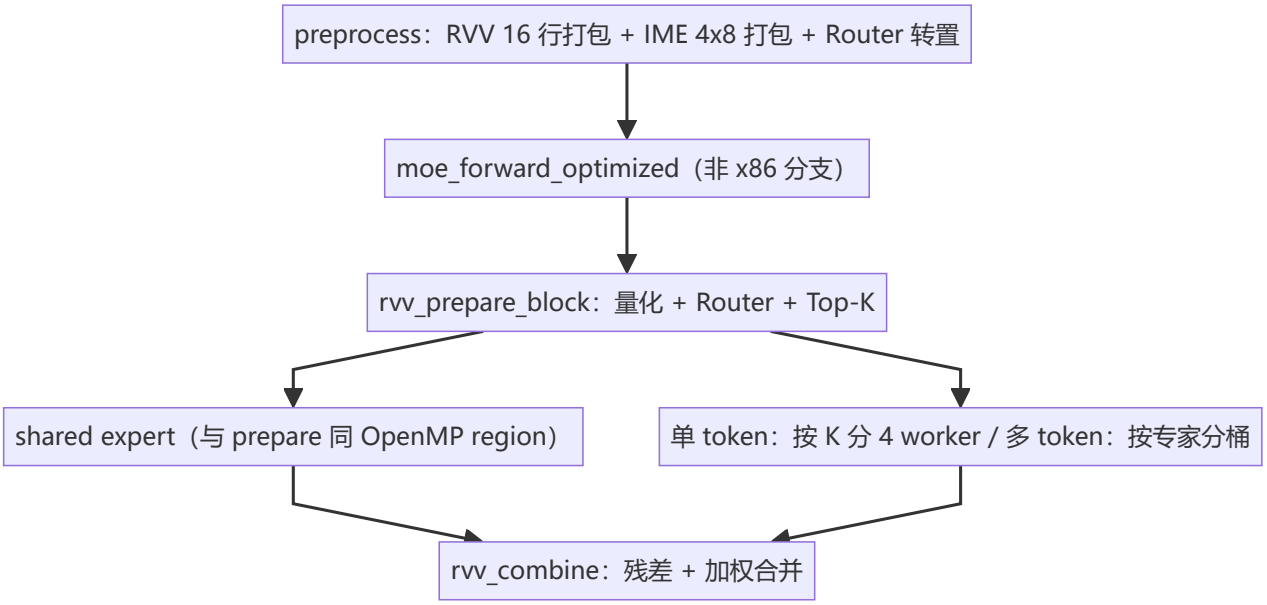
它没有增加矩阵指令、权重读取或累加器数量，只是把下一 K 块的 load 提前. 该 leaf 只在运行时形状精确匹配 S2（`t_count == 1 && d_model == 1024 && d_ff == 512`）时使用，其他形状继续走原来的 8 展开内核. 同一二进制、同一节点交替跑五组 A/B，中位时间从 0.47536 s 降到 0.45847 s，约 **3.55%**——和 gprof 的结论一致：收益来自占比 63% 的 IME leaf，而不是只占 0.22% 的 Router.

过程中还验证了两个"看起来该做、实测是负优化"的实验：

- **shared expert 4 核并行**：把 shared 的 output tile 分给 4 个 worker（一版用 IME Gate/Up/Down + SwiGLU 中间串行段，一版用 16-output RVV 微内核按 output block 并行），300 轮时间 0.924--0.985 s，几乎翻倍. 原因不是数学工作量，而是额外创建了一个很短的 OpenMP region、在 Gate/Up、SwiGLU、Down 之间引入多次 barrier，四核并发读同一份大权重流还加剧了 IME 执行资源和内存带宽竞争——原来的 routed 并行已经提供了最合适的粗粒度；
- **K 循环展开因子**：8 展开改 16（0.4761 s）和 4（约 0.479 s）都没有收益，说明热点不是循环分支本身，8 是 K60 前端代码体积和循环控制之间较好的平衡.

## 1.5 分场景调度与最终结果

调度逻辑（`rvv_forward`）和 x86 主线一致，按场景特征分路：




- 单 token：shared expert 先串行算完，4 个 routed expert 用 `#pragma omp parallel for schedule(static)` 按 K 直接分给 4 个 worker（遍历所有 expert 的 static 分块可能把几个选中 expert 压到同一个 worker，按 K 分保证四个路由一人一个）；
- 多 token：`rvv_prepare_block`（量化 + Router + Top-K）和 shared expert 合成**同一个** OpenMP region——两者对 token 块的所有权完全一致，一次 fork/barrier 就省掉了；然后 `rvv_build_buckets` 按专家分桶，每个专家只处理自己的 token 子集（`#pragma omp parallel for` 按专家并行，每个 assignment 恰好出现在一个桶里，输出行天然不相交，不需要同步），最后 `rvv_combine` 加权合并；
- `preprocess` 里 `OMP_WAIT_POLICY=ACTIVE`、`OMP_PROC_BIND=close`、`omp_set_num_threads(4)`，避免这些亚毫秒级并行区反复唤醒/迁移线程.

RISC-V 节点上的完整演进（baseline：S1 0.395 s、S2 6.10 s、S3 25.3 s、S4 33.1 s，都是同一节点上的框架参考实现）：

| 阶段 | S1 | S2 | S3 | S4 | 日志 |
|---|---|---|---|---|---|
| 标量移植（OpenMP 按 token 并行） | 4.06x | 4.84x | 4.20x | 2.55x | `20260803_085912.log` |
| RVV v1 | 3.67x | 3.71x | 42.96x | 30.13x | `20260803_113845.log` |
| + IME | 12.09x | 13.88x | 45.98x | 27.78x | `20260804_143119.log` |
| 最终版（双输出 + S2 软件流水） | 14.49x | 19.81x | 113.18x | 64.44x | `20260805_052125.log` |

两个值得展开的地方：

1. **RVV v1 的 S1/S2 是回退的**（3.67x/3.71x，比标量移植的 4.06x/4.84x 还低）. RVV v1 是按 S3/S4 的 batch 形状写的，单 token 场景里向量化的固定开销（vsetvl、打包、清零）反而比收益大——和主线里"矩阵化在 S3 掉到 2.82x"是同一个教训：**优化形状和问题形状要匹配**，S1/S2 这种单 token 小专家场景要的是降低单条指令的延迟而不是堆吞吐. IME 的 M=4 tile 天然为小 batch 设计，换上去之后单 token 直接回到 12x 以上；
2. **S4 超额完成**：当初的预期是 27x（带宽约束，见 `moe_riscv_rvv_optimization.md` 里的估算表），实际跑到 64.44x. 打包 + 分桶让权重按 token 组复用、DRAM 流量降 8 倍之后，S4 从"带宽瓶颈"变成了"计算瓶颈"，向量化的收益全部兑现. 对照当初定的目标（16.0 / 18.5 / 114.5 / 27.0），S2、S4 超额，S3 基本打平，S1 差约 10%——单 token 微秒级的路径本来就受频率和线程状态影响，属于意料之中.